AutoPass-Gen: Scenario Generation for Evaluating Urgency-Aware Passing

Rohan Sachdeva Xinwei Mai Pranav Prabu Rathang Pandit

UC San Diego

One-Sentence Project Framing

We are not trying to build the best driver.

We are building a system that finds when driving policies fail.

- Maneuver: **passing a slow lead vehicle**
- Stressor: **urgency and deadlines**
- Output: **failure cases, traces, and metrics**

Problem

- Standard autonomous driving evaluation often uses:
  - fixed routes,
  - random traffic,
  - or a few hand-designed scenarios.

Problem

- Standard autonomous driving evaluation often uses:
  - fixed routes,
  - random traffic,
  - or a few hand-designed scenarios.
- These evaluations can miss rare but important failures.

Problem

- Standard autonomous driving evaluation often uses:
  - fixed routes,
  - random traffic,
  - or a few hand-designed scenarios.
- These evaluations can miss rare but important failures.
- Our question:

When does urgency make a passing policy unsafe, overly conservative, or inconsistent?

Why Passing?

- Passing is narrow enough to implement, but rich enough to expose real decision failures.

Why Passing?

- Passing is narrow enough to implement, but rich enough to expose real decision failures.
- The agent must reason about:
  - slow lead vehicle,
  - rear traffic,
  - oncoming gap,
  - visibility and occlusion,
  - deadline pressure.

Why Passing?

- Passing is narrow enough to implement, but rich enough to expose real decision failures.
- The agent must reason about:
  - slow lead vehicle,
  - rear traffic,
  - oncoming gap,
  - visibility and occlusion,
  - deadline pressure.
- The interesting behavior is at the boundary:
  - pass safely,
  - wait too long,
  - replan,
  - or pass unsafely.

What Makes This Project Different

Typical Driving Project	**AutoPass-Gen**
Build a better driving policy	Generate scenarios that expose failures
Evaluate on fixed scenarios	Mutate scenarios toward harder cases
Focus on perception/control alone	Focus on safety-efficiency decision boundary
Only measure success rate	Log interpretable failure traces

The generator is the research object.

Current Status

- Week 1 local prototype is complete.

Current Status

- Week 1 local prototype is complete.
- Implemented full simulator-independent loop:
  - scenario generation,
  - urgency interpretation,
  - pass/wait/replan policy,
  - safety checker,
  - execution backend,
  - evaluator,
  - traces and plots.

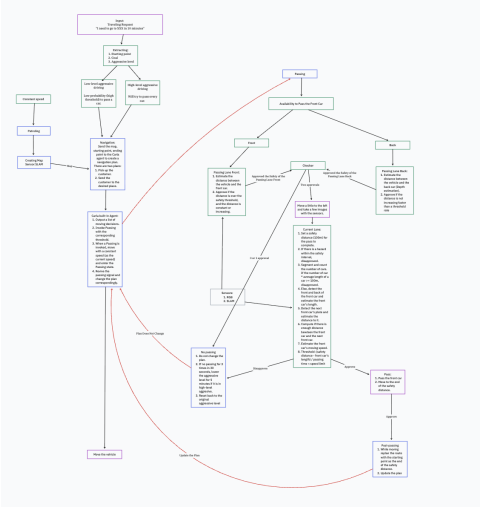
Current Status

- Week 1 local prototype is complete.
- Implemented full simulator-independent loop:
  - scenario generation,
  - urgency interpretation,
  - pass/wait/replan policy,
  - safety checker,
  - execution backend,
  - evaluator,
  - traces and plots.
- Important framing:

This is not yet the full CARLA prototype.

It is the integration layer before CARLA.

System Architecture



Implemented Prototype Loop

`ScenarioSpec`

↓

Request/Urgency Interpreter

↓

Perception/Map State Estimate

↓

Passing Agent: `pass` / `wait` / `replan`

↓

Safety Checker

↓

Execution Backend

↓

Evaluator: metrics + trace + failure type

Same interfaces will be reused for CARLA.

Scenario Representation

- Each generated case is a structured object, not a one-off script.

$$\{\texttt{request}, \texttt{route}, \texttt{ego}, \texttt{lead}, \texttt{rear}, \texttt{oncoming}, \texttt{occlusion}, \texttt{weather}, \texttt{sensor}\}$$

- This lets us control:
  - deadlines,
  - lead vehicle speed,
  - rear traffic,
  - oncoming gap,
  - visibility,
  - weather and sensor mode.

Passing Decision

- At each timestep, the agent estimates:
  - front distance,
  - rear distance,
  - oncoming distance,
  - visibility,
  - urgency and delay cost.

Passing Decision

- At each timestep, the agent estimates:
  - front distance,
  - rear distance,
  - oncoming distance,
  - visibility,
  - urgency and delay cost.
- Policy proposes:

`pass` `wait` `replan`

Passing Decision

- At each timestep, the agent estimates:
  - front distance,
  - rear distance,
  - oncoming distance,
  - visibility,
  - urgency and delay cost.
- Policy proposes:

```
pass   wait   replan
```

- Safety checker can reject unsafe pass proposals.

Safety Constraint

$$d_{\mathrm{gap}} > v_{\mathrm{ego}} t_{\mathrm{pass}} + \frac{1}{2} a_{\mathrm{ego}} t_{\mathrm{pass}}^2 + \Delta$$

- Urgency can increase willingness to consider passing.
- It cannot override:
  - low TTC,
  - unsafe rear traffic,
  - poor visibility,
  - insufficient oncoming gap.

Policies Compared

- **No-pass baseline**
  - never passes,
  - exposes over-conservative delay.

Policies Compared

- **No-pass baseline**
  - never passes,
  - exposes over-conservative delay.
- **Aggressive baseline**
  - pursues progress,
  - exposes unsafe passing.

Policies Compared

- **No-pass baseline**
  - never passes,
  - exposes over-conservative delay.
- **Aggressive baseline**
  - pursues progress,
  - exposes unsafe passing.
- **AutoPass**
  - urgency-aware,
  - attempts selected passes,
  - safety checker rejects risky proposals.

Initial Local Results: 50 Scenarios

Policy	Failures	Avg. time (s)	Pass attempts	Unsafe passes
No-pass	16/50	96.58	0	0
AutoPass	13/50	90.16	19	0
Aggressive	41/50	60.08	154	134

- Aggressive is fastest but unsafe.
- No-pass is safe but slow.
- AutoPass is the intended middle behavior.

Failure Type Breakdown

Policy	None	Delay	Missed safe pass	Unsafe pass
No-pass	34	16	0	0
AutoPass	37	11	2	0
Aggressive	9	2	0	39

- AutoPass reduces delay failures relative to no-pass.
- AutoPass avoids unsafe passes in this local evaluation.
- Aggressive fails mostly through unsafe passing.

What We Learned From the Prototype

- The architecture runs end-to-end.
- The traces are useful for debugging and explaining decisions.
- Generated scenarios already expose meaningful behavior:
  - conservative delay,
  - unsafe passing,
  - missed safe opportunities.

What We Learned From the Prototype

- The architecture runs end-to-end.
- The traces are useful for debugging and explaining decisions.
- Generated scenarios already expose meaningful behavior:
  - conservative delay,
  - unsafe passing,
  - missed safe opportunities.
- We also fixed two local backend issues:
  - lane-aware collision logic,
  - safe car-following for wait/replan actions.

Honest Limitation

Current results are local prototype results, not full CARLA results.

- Local backend uses lightweight kinematics.
- It validates interfaces and the closed-loop evaluation logic.
- Next step is replacing world state and execution with CARLA:
  - actor spawning,
  - sensors,
  - map routes,
  - vehicle controls.

Week 2 Plan: Minimal CARLA Integration

- Take one `ScenarioSpec`.
- Spawn:
  - ego vehicle,
  - slow lead vehicle,
  - rear vehicle,
  - oncoming vehicle.

Week 2 Plan: Minimal CARLA Integration

- Take one `ScenarioSpec`.
- Spawn:
  - ego vehicle,
  - slow lead vehicle,
  - rear vehicle,
  - oncoming vehicle.
- Set route, weather, and sensor configuration.
- Run one pass/wait/replan rollout.
- Save the same trace and metrics format.

Goal: one working CARLA-backed passing scenario.

Team Responsibilities

- **Rohan:** integration, evaluation, plots, report, demo script.
- **Xinwei:** passing logic, safety checker, decision-diagram alignment.
- **Pranav:** CARLA actor spawning, routes, weather, sensors.
- **Rathang:** perception/map tools, baselines, metrics, trace examples.

Shared contract: `ScenarioSpec` **in,** `PassState`**/metrics out.**

What We Want Feedback On

- Is the current simulator-independent prototype a reasonable Week 1 checkpoint?
- Is the CARLA integration plan scoped correctly?
- Should we prioritize:
  - stronger scenario generation,
  - more realistic CARLA execution,
  - or RGB/depth perception modules?
- Are the failure types appropriate for the final report?

Closing

AutoPass-Gen turns passing into a stress-test problem.

The goal is not just to drive, but to find the boundary where driving policies fail.